

# Assignment I - Compiling a Custom Linux Kernel & Implementing New System Calls

313512009 施丞蔚

## I. Compiling a custom Linux Kernel

- Include screenshot of the outputs of `uname -a` and `cat /proc/version` to prove that you compiled the kernel by yourself.

```
~ # uname -a
Linux (none) 6.1.0-os-313512009 #1 SMP Thu Oct 16 13:52:21 UTC 2025 riscv64 GNU/Linux
~ # cat /proc/version
Linux version 6.1.0-os-313512009 (ubuntu@ecb5a3b62549) (riscv64-linux-gnu-gcc (Ubuntu 13.3.0-6ubuntu2~24.04) 13.3.0, GNU ld (GNU Binutils for Ubuntu) 2.42) #1 SMP Thu Oct 16 13:52:21 UTC 2025
```

- List the steps you took to compile the kernel

### 1. 取得 v6.1 LTS 原始碼

```
ubuntu@ecb5a3b62549:~$ git clone https://github.com/torvalds/linux --branch v6.1 --depth 1 linux
Cloning into 'linux'...
remote: Enumerating objects: 83369, done.
remote: Counting objects: 100% (83369/83369), done.
remote: Compressing objects: 100% (75828/75828), done.
remote: Total 83369 (delta 6741), reused 80727 (delta 6689), pack-reused 0 (from 0)
Receiving objects: 100% (83369/83369), 234.73 MiB | 6.08 MiB/s, done.
Resolving deltas: 100% (6741/6741), done.
Note: switching to '830b3c68c1fb1e9176028d02ef86f3cf76aa2476'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

    git switch -c <new-branch-name>

Or undo this operation with:

    git switch -

Turn off this advice by setting config variable advice.detachedHead to false
```

### 2. 產生 RISC-V 預設設定 (defconfig)

```
ubuntu@ecb5a3b62549:~$ cd linux
ubuntu@ecb5a3b62549:~/linux$ make ARCH=riscv CROSS_COMPILE=riscv64-linux-gnu- defconfig
HOSTCC scripts/basic/fixdep
HOSTCC scripts/kconfig/conf.o
HOSTCC scripts/kconfig/confdata.o
HOSTCC scripts/kconfig/expr.o
LEX scripts/kconfig/lexer.lex.c
YACC scripts/kconfig/parser.tab.[ch]
HOSTCC scripts/kconfig/lexer.lex.o
HOSTCC scripts/kconfig/menu.o
HOSTCC scripts/kconfig/parser.tab.o
HOSTCC scripts/kconfig/preprocess.o
HOSTCC scripts/kconfig/symbol.o
HOSTCC scripts/kconfig/util.o
HOSTLD scripts/kconfig/conf
*** Default configuration is based on 'defconfig'
#
# configuration written to .config
#
```

### 3. 設定本地版本字尾, 作為自編證明

```
ubuntu@ecb5a3b62549:~/linux$ ./scripts/config --set-str CONFIG_LOCALVERSION "-os-313512009"
ubuntu@ecb5a3b62549:~/linux$ ./scripts/config --disable CONFIG_LOCALVERSION_AUTO
ubuntu@ecb5a3b62549:~/linux$ make ARCH=riscv CROSS_COMPILE=riscv64-linux-gnu- olddefconfig
```

4. 在正式編譯前, 先檢查版本字串

```
ubuntu@ecb5a3b62549:~/linux$ make ARCH=riscv CROSS_COMPILE=riscv64-linux-gnu- kernelrelease
6.1.0-os-313512009
```

5. 編譯 Kernel

```
ubuntu@ecb5a3b62549:~/linux$ make -j$(nproc)" ARCH=riscv CROSS_COMPILE=riscv64-linux-gnu-
DTC      arch/riscv/boot/dts/microchip/mpfs-icicle-kit.dtb
DTC      arch/riscv/boot/dts/microchip/mpfs-m100pfsevp.dtb
DTC      arch/riscv/boot/dts/sifive/hifive-unleashed-a00.dtb
DTC      arch/riscv/boot/dts/starfive/jh7100-beaglev-starlight.dtb
DTC      arch/riscv/boot/dts/microchip/mpfs-polarberry.dtb
```

...

```
OBJCOPY arch/riscv/boot/Image
GZIP     arch/riscv/boot/Image.gz
Kernel: arch/riscv/boot/Image.gz is ready
```

6. 用 TA 提供的 initramfs 以 QEMU 開機驗證

```
ubuntu@ecb5a3b62549:~$ qemu-system-riscv64 -nographic -machine virt \
-kernel ~/linux/arch/riscv/boot/Image \
-initrd ~/initramfs.cpio.gz \
-append "console=ttyS0 loglevel=3"
```

OpenSBI v1.3

OpenSBI

```
Platform Name      : riscv-virtio,qemu
Platform Features  : medeleg
Platform HART Count : 1
Platform IPI Device : aclint-mswi
Platform Timer Device : aclint-mtimer @ 100000000Hz
Platform Console Device : uart8250
Platform HSM Device : ---
Platform PMU Device : ---
Platform Reboot Device : sifive_test
Platform Shutdown Device : sifive_test
Platform Suspend Device : ---
Platform CPPC Device : ---
Firmware Base      : 0x80000000
Firmware Size      : 322 KB
Firmware RW Offset : 0x40000
Firmware RW Size    : 66 KB
Firmware Heap Offset : 0x48000
Firmware Heap Size  : 34 KB (total), 2 KB (reserved), 9 KB (used), 22 KB (free)
Firmware Scratch Size : 4096 B (total), 760 B (used), 3336 B (free)
Runtime SBI Version : 1.0

Domain0 Name       : root
Domain0 Boot HART  : 0
Domain0 HARTs      : 0*
Domain0 Region00   : 0x0000000020000000-0x00000000200ffff M: (I,R,W) S/U: ()
Domain0 Region01   : 0x0000000080040000-0x000000008005ffff M: (R,W) S/U: ()
Domain0 Region02   : 0x0000000080000000-0x000000008003ffff M: (R,X) S/U: ()
Domain0 Region03   : 0x0000000000000000-0xffffffff M: (R,W,X) S/U: (R,W,X)
Domain0 Next Address : 0x0000000080200000
Domain0 Next Arg1   : 0x0000000087e00000
Domain0 Next Mode    : S-mode
Domain0 SysReset     : yes
Domain0 SysSuspend   : yes

Boot HART ID       : 0
Boot HART Domain    : root
Boot HART Priv Version : v1.12
Boot HART Base ISA   : rv64imafdc
Boot HART ISA Extensions : time,sstc
Boot HART PMP Count  : 16
Boot HART PMP Granularity : 4
Boot HART PMP Address Bits : 54
Boot HART MHPM Count : 16
Boot HART MIDELEG    : 0x0000000000001666
Boot HART MEDELEG    : 0x000000000f0b509
```

## 7. 在 QEMU 內輸出版本資訊並截圖

```
~ # uname -a
Linux (none) 6.1.0-os-313512009 #1 SMP Thu Oct 16 13:52:21 UTC 2025 riscv64 GNU/Linux
~ # cat /proc/version
Linux version 6.1.0-os-313512009 (ubuntu@ecb5a3b62549) (riscv64-linux-gnu-gcc (Ubuntu 13.3.0-6ubuntu2-24.04) 13.3.0, GNU ld (GNU Binutils for Ubuntu) 2.42) #1 SMP Thu Oct 16 13:52:21 UTC 2025
```

### ● Answer the Q&As

#### 1. What are the main differences between the RISC-V and x86 architectures?

- 指令集哲學: RISC-V 是開放、模組化的 RISC (基礎 ISA + 可選延伸 M/A/F/D/C/V...); x86 屬於封閉且歷史悠久的 CISC, 指令長度可變、相容性包袱重。
- 特權與開機機制: RISC-V 常見 M/S/U 三層, 早期由 OpenSBI 提供 SBI 服務再進入作業系統; x86 走 BIOS/UEFI → 實模式/保護模式/長模式的傳統序列。
- ABI / 呼叫慣例: RISC-V (LP64D) 以 a0–a7 傳遞參數/回傳值; x86-64 是 SysV AMD64 (rdi, rsi, rdx, rcx, r8, r9...).
- 平台專屬子系統: 中斷控制器、頁表格式、啟動碼、系統呼叫號碼表與彙編巨集等都各自不同 (Linux 原始碼分別位於 arch/riscv/ 與 arch/x86/).
- 生態定位: RISC-V 在開放硬體、嵌入式/SoC、學研很常見; x86 主導 PC/伺服器市場。

#### 2. Why do the architecture differences matter when building the kernel? What happens if you build the kernel without the correct RISC-V cross-compilation flag?

- 重要原因
  - Kbuild 需依 ARCH=riscv 選對 架構專屬原始碼與設定 (如 arch/riscv/ 的啟動碼、頁表、SBI 介面...).
  - Toolchain 需用 CROSS\_COMPILE=riscv64-linux-gnu- 產生 riscv64 ELF, 且符合 RISC-V 的 ABI/匯編語法/指令集延伸。
- 若旗標錯誤或缺少
  - 會編出 x86\_64 (或其他架構) 的核心或物件檔, QEMU 以 RISC-V 啟動時直接 非法指令/無法開機。
  - 就算勉強連過, 系統呼叫號碼、啟動流程、頁表格式 等不對也會在早期初始化就 Kernel panic。
  - 使用錯誤 ABI 會導致 呼叫慣例不匹配 (參數/回傳值暫存器不同), 出現難以追蹤的崩潰或錯誤行為。

**3. Why is Docker used in this assignment, and what advantages does it provide? Please list at least two of them.**

- 可重現性:把交叉編譯器、QEMU、套件版本 固定 在容器中, 避免「我這台可以、你那台不行」之類的情況。
- 環境隔離:不汙染主機系統(不用在主機安裝一堆工具鏈), 清理/重建環境很容易。
- 可攜與一致:助教/同學在不同機器只要跑同一份 docker-compose.yml 就能得到相同開發環境。
- 效能/資源控管:能限制與觀察容器資源(CPU/記憶體/IO), 便於大量編譯時管理。

## II. Implementing new System Calls

- Include four screenshots of the outputs

1. The execution result of provided `test_revstr` user program for `sys_revstr`

```
~ # /usr/bin/test_revstr
Ori: hello
Rev: olleh
Ori: Operating System
Rev: metsyS gnitarep0
```

2. The `dmesg` output of `sys_revstr`

```
~ # dmesg | tail -n 40
[ 0.288837] printk: console [ttyS0] enabled
[ 0.301825] loop: module loaded
[ 0.304733] e1000e: Intel(R) PRO/1000 Network Driver
[ 0.304748] e1000e: Copyright(c) 1999 - 2015 Intel Corporation.
[ 0.306077] usbcore: registered new interface driver uas
[ 0.306202] usbcore: registered new interface driver usb-storage
[ 0.306765] mousedev: PS/2 mouse device common for all mice
[ 0.309323] goldfish_rtc 101000.rtc: registered as rtc0
[ 0.309873] goldfish_rtc 101000.rtc: setting system clock to 2025-10-17T11:23:48 UTC (1760700228)
[ 0.311848] syscon-poweroff poweroff: pm_power_off already claimed for sbi_srst_power_off
[ 0.312248] syscon-poweroff: probe of poweroff failed with error -16
[ 0.313221] sdhci: Secure Digital Host Controller Interface driver
[ 0.313234] sdhci: Copyright(c) Pierre Ossman
[ 0.313379] sdhci-pltfm: SDHCI platform and OF driver helper
[ 0.313810] usbcore: registered new interface driver usbhid
[ 0.313828] usbhid: USB HID core driver
[ 0.314036] riscv-pmu-sbi: SBI PMU extension is available
[ 0.314400] riscv-pmu-sbi: 16 firmware and 18 hardware counters
[ 0.314423] riscv-pmu-sbi: Perf sampling/filtering is not supported as sscof extension is not available
[ 0.316532] NET: Registered PF_INET6 protocol family
[ 0.321799] Segment Routing with IPv6
[ 0.321939] In-situ OAM (IOAM) with IPv6
[ 0.322242] sit: IPv6, IPv4 and MPLS over IPv4 tunneling driver
[ 0.324470] NET: Registered PF_PACKET protocol family
[ 0.325362] 9pnet: Installing 9P2000 support
[ 0.325567] Key type dns_resolver registered
[ 0.325625] start plist test
[ 0.329247] end plist test
[ 0.330749] debug_vm_pgtable: [debug_vm_pgtable]: Validating architecture page table helpers
[ 0.369769] Freeing unused kernel image (initmem) memory: 2176K
[ 0.377921] Run /init as init process
[ 0.377947] with arguments:
[ 0.377962] /init
[ 0.377980] with environment:
[ 0.377994] HOME=/
[ 0.378007] TERM=linux
[ 57.934606] The origin string: hello
[ 57.934673] The reversed string: olleh
[ 57.935103] The origin string: Operating System
[ 57.935114] The reversed string: metsyS gnitarep0
~ #
```

3. The execution result of provided `test_templibuf` user program for `sys_templibuf`

```
~ # /usr/bin/test_templibuf
Hello Operating Systems
Operating Systems
```

#### 4. The dmesg output of sys\_temptbuf

```
~ # dmesg | tail -n 80
[ 0.136525] TCP: Hash tables configured (established 1024 bind 1024)
[ 0.137564] UDP hash table entries: 256 (order: 2, 24576 bytes, linear)
[ 0.137808] UDP-Lite hash table entries: 256 (order: 2, 24576 bytes, linear)
[ 0.138819] NET: Registered PF_UNIX/PF_LOCAL protocol family
[ 0.141444] RPC: Registered named UNIX socket transport module.
[ 0.141486] RPC: Registered udp transport module.
[ 0.141495] RPC: Registered tcp transport module.
[ 0.141502] RPC: Registered tcp NFSv4.1 backchannel transport module.
[ 0.141631] PCI: CLS 0 bytes, default 64
[ 0.147704] Unpacking initramfs...
[ 0.150813] workingset: timestamp_bits=46 max_order=15 bucket_order=0
[ 0.170966] NFS: Registering the id_resolver key type
[ 0.170889] Key type id_resolver registered
[ 0.170920] Key type id_legacy registered
[ 0.171231] nfs4FileLayout_init: NFSv4 File Layout Driver Registering...
[ 0.171312] nfs4FlexFileLayout_init: NFSv4 Flexfile Layout Driver Registering...
[ 0.171746] 9p: Installing v9fs 9p2000 file system support
[ 0.173341] NET: Registered PF_ALG protocol family
[ 0.173589] Block layer SCSI generic (bsg) driver version 0.4 loaded (major 250)
[ 0.173711] io scheduler mq-deadline registered
[ 0.173791] io scheduler kyber registered
[ 0.181274] pci-host-generic 30000000.pci: host bridge /soc/pci@30000000 ranges:
[ 0.181978] pci-host-generic 30000000.pci: IO 0x0003000000..0x000300ffff -> 0x0000000000
[ 0.182381] pci-host-generic 30000000.pci: MEM 0x0040000000..0x007fffffff -> 0x0040000000
[ 0.182437] pci-host-generic 30000000.pci: MEM 0x0400000000..0x07fffffff -> 0x0400000000
[ 0.182892] pci-host-generic 30000000.pci: Memory resource size exceeds max for 32 bits
[ 0.184229] pci-host-generic 30000000.pci: ECAM at [mem 0x30000000-0x3fffffff] for [bus 00-ff]
[ 0.185315] pci-host-generic 30000000.pci: PCI host bridge to bus 0000:00
[ 0.185546] pci_bus 0000:00: root bus resource [bus 00-ff]
[ 0.185625] pci_bus 0000:00: root bus resource [io 0x0000-0xffff]
[ 0.185667] pci_bus 0000:00: root bus resource [mem 0x40000000-0x7fffffff]
[ 0.185676] pci_bus 0000:00: root bus resource [mem 0x40000000-0x7fffffff]
[ 0.186808] pci 0000:00:00.0: [1b36:0008] type 00 class 0x060000
[ 0.265346] Freeing initrd memory: 1800K
[ 0.278286] Serial: 8250/16550 driver, 4 ports, IRQ sharing disabled
[ 0.285189] printk: console [ttyS0] disabled
[ 0.286836] 10000000.serial: ttyS0 at MMIO 0x10000000 (irq = 1, base_baud = 230400) is a 16550A
[ 0.288837] printk: console [ttyS0] enabled
[ 0.301825] loop: module loaded
[ 0.304733] e1000e: Intel(R) PRO/1000 Network Driver
[ 0.304748] e1000e: Copyright(c) 1999 - 2015 Intel Corporation.
[ 0.306077] usbcore: registered new interface driver uas
[ 0.306202] usbcore: registered new interface driver usb-storage
[ 0.306765] mousedev: PS/2 mouse device common for all mice
[ 0.309323] goldfish_rtc 101000.rtc: registered as rtc0
[ 0.309873] goldfish_rtc 101000.rtc: setting system clock to 2025-10-17T11:23:48 UTC (1760700228)
[ 0.311848] syscon-poweroff poweroff: pm_power_off already claimed for sbi_srst_power_off
[ 0.312248] syscon-poweroff: probe of poweroff failed with error -16
[ 0.313221] sdhci: Secure Digital Host Controller Interface driver
[ 0.313234] sdhci: Copyright(c) Pierre Ossman
[ 0.313379] sdhci-pltfm: SDHCI platform and OF driver helper
[ 0.313810] usbcore: registered new interface driver usbhid
[ 0.313828] usbhid: USB HID core driver
[ 0.314036] riscv-pmu-sbi: SBI PMU extension is available
[ 0.314400] riscv-pmu-sbi: 16 firmware and 18 hardware counters
[ 0.314423] riscv-pmu-sbi: Perf sampling/filtering is not supported as sscof extension is not available
[ 0.316532] NET: Registered PF_INET6 protocol family
[ 0.321799] Segment Routing with IPv6
[ 0.321939] In-situ OAM (IOAM) with IPv6
[ 0.322242] sit: IPv6, IPv4 and MPLS over IPv4 tunneling driver
[ 0.324470] NET: Registered PF_PACKET protocol family
[ 0.325362] 9pnet: Installing 9p2000 support
[ 0.325567] Key type dns_resolver registered
[ 0.325625] start plist test
[ 0.329247] end plist test
[ 0.330749] debug_vm_pgtable: [debug_vm_pgtable]: Validating architecture page table helpers
[ 0.369769] Freeing unused kernel image (initmem) memory: 2176K
[ 0.377921] Run /init as init process
[ 0.377947] with arguments:
[ 0.377962] /init
[ 0.377980] with environment:
[ 0.377994] HOME=/
[ 0.378007] TERM=linux
[ 57.934606] The origin string: hello
[ 57.934673] The reversed string: olleh
[ 57.935103] The origin string: Operating System
[ 57.935114] The reversed string: metsyS gnitarepO
[ 192.697350] [temptbuf] Added: Hello
[ 192.697423] [temptbuf] Added: Operating Systems
[ 192.698905] [temptbuf] Removed: Hello
#
```

- Explain how you added system calls to your custom Linux kernel and the implementation details of `sys_revstr` and `sys_tempbuf`

### 1. 新增內核檔案 `kernel/os_hw1_syscalls.c`

```
// kernel/os_hw1_syscalls.c
#include <linux/kernel.h>
#include <linux/syscalls.h>
#include <linux/slab.h>
#include <linux/uaccess.h>
#include <linux/list.h>
#include <linux/mutex.h>
#include <linux/string.h>
#include <linux/errno.h>

/*
 * sys_revstr (nr = 451)
 * 參數: (char __user *str, size_t n)
 * 動作: 印出原字串 -> 反轉 -> 印出反轉字串 -> 寫回 user 空間 -> 回傳 0
 * 題目說可不特別處理無效引數, 但 copy_from/to_user 失敗時回 -EFAULT
 */
asmlinkage long __riscv_sys_revstr(char __user *str, size_t n)
{
    char *kbuf;
    size_t i = 0, j;

    if (!str || n == 0)
        return 0;

    kbuf = kmalloc(n + 1, GFP_KERNEL);
    if (!kbuf)
        return -ENOMEM;

    if (copy_from_user(kbuf, str, n)) {
        kfree(kbuf);
        return -EFAULT;
    }
    kbuf[n] = '\0';

    pr_info("The origin string: %s\n", kbuf);

    j = n - 1;
    while (i < j) {
        char t = kbuf[i];
        kbuf[i] = kbuf[j];
        kbuf[j] = t;
        i++;
        j--;
    }

    pr_info("The reversed string: %s\n", kbuf);

    if (copy_to_user(str, kbuf, n)) {
        kfree(kbuf);
        return -EFAULT;
    }

    kfree(kbuf);
    return 0;
}

/*
 * sys_tempbuf (nr = 452)
 * 參數: (int mode, void __user *data, size_t size)
 * 模式:
 *   ADD    : 加到 list 末端, 印 [tempbuf] Added: <data>, 成功回 0
 *   REMOVE : 移除第一個匹配, 印 [tempbuf] Removed: <data>; 找不到回 -ENOENT
 *   PRINT  : 以空白串接所有字串, 複製回 data (最多 size bytes), 回傳實際複製字元數 (不含 '\0')
 * 失敗:
 *   -EFAULT: user 指標無效 (data=NULL / size=0 / copy_from/to_user 失敗)
 *   註: 題目保證 PRINT 產出長度不會超過 512 bytes。
 */
enum mode { PRINT = 0, ADD = 1, REMOVE = 2 };

struct tb_node {
    struct list_head node;
    char *s;
    size_t len;
};

static LIST_HEAD(tb_head);
static DEFINE_MUTEX(tb_lock);

asmlinkage long __riscv_sys_tempbuf(int mode, void __user *data, size_t size)
{
    if (mode != ADD && mode != REMOVE && mode != PRINT)
        return -EINVAL;

    /* ADD / REMOVE: 先把 user 的字串複製進 kernel */
    if (mode == ADD || mode == REMOVE) {
        char *kstr;

        if (!data || size == 0)
            return -EFAULT;

        kstr = kmalloc(size + 1, GFP_KERNEL);
        if (!kstr)
            return -ENOMEM;

        if (copy_from_user(kstr, data, size)) {
            kfree(kstr);
            return -EFAULT;
        }
        kstr[size] = '\0';

        if (mode == ADD) {
            struct tb_node *n = kmalloc(sizeof(*n), GFP_KERNEL);
            if (!n) {
                kfree(kstr);
                return -ENOMEM;
            }
            n->s = kstr;
            n->len = size;

            mutex_lock(&tb_lock);
            list_add_tail(&n->node, &tb_head);
            mutex_unlock(&tb_lock);
        }
    }
}
```

```

        pr_info("[tempbuf] Added: %s\n", kstr);
        return 0;
    } else { /* REMOVE */
        struct tb_node *pos, *tmp;
        int found = 0;

        mutex_lock(&tb_lock);
        list_for_each_entry_safe(pos, tmp, &tb_head, node) {
            if (pos->len == size && !memcmp(pos->s, kstr, size)) {
                list_del(&pos->node);
                pr_info("[tempbuf] Removed: %s\n", pos->s);
                kfree(pos->s);
                kfree(pos);
                found = 1;
                break;
            }
        }
        mutex_unlock(&tb_lock);

        kfree(kstr);
        return found ? 0 : -ENOENT;
    }
}

/* PRINT: 串接 list 內容, 以單一空白分隔, 拷回 user, 回傳拷回的實際字元數 (不含 '\0') */
if (mode == PRINT) {
    size_t sum = 0, cnt = 0, need, cap, written = 0;
    struct tb_node *pos;
    char *out;

    if (!data || size == 0)
        return -EFAULT;

    /* 第一次遍歷計算總長度 (字串總長 + 空白數) */
    mutex_lock(&tb_lock);
    list_for_each_entry(pos, &tb_head, node) {
        sum += pos->len;
        cnt++;
    }
    mutex_unlock(&tb_lock);

    if (cnt == 0) {
        /* 空清單: 回傳空字串 */
        char zero = '\0';
        if (copy_to_user(data, &zero, 1))
            return -EFAULT;
        return 0;
    }

    need = sum + (cnt - 1);
    cap = (need > 512) ? 512 : need; /* 題目保證不超過 512, 這裡仍防呆 */

    out = kmalloc(cap + 1, GFP_KERNEL);
    if (!out)
        return -ENOMEM;

    /* 第二次遍歷實際拷貝, 注意容量 cap */
    {
        size_t idx = 0;
        size_t i = 0;

        mutex_lock(&tb_lock);
        list_for_each_entry(pos, &tb_head, node) {
            if (i++ > 0) {
                if (idx < cap)
                    out[idx++] = ' ';
            }
            if (idx < cap) {
                size_t can = pos->len;
                if (can > cap - idx)
                    can = cap - idx;
                if (can) {
                    memcpy(out + idx, pos->s, can);
                    idx += can;
                }
            }
        }
        mutex_unlock(&tb_lock);

        out[idx] = '\0';
        written = idx; /* 不含 '\0' */
    }

    /* 連同結尾 '\0' 一起拷回 (但回傳值不含 '\0') */
    if (copy_to_user(data, out, (written + 1 <= size) ? (written + 1) : size)) {
        kfree(out);
        return -EFAULT;
    }

    kfree(out);
    return (int)written;
}

return 0;
}

```

## 2. 讓 Kbuild 編進核心 (修改 kernel/Makefile)

```

ubuntu@ecb5a3b62549:~/linux$ grep -q 'os_hw1_syscalls.o' kernel/Makefile || echo 'obj-y += os_hw1_syscalls.o' >> kernel/Makefile
ubuntu@ecb5a3b62549:~/linux$ grep 'os_hw1_syscalls.o' -n kernel/Makefile
162:obj-y += os_hw1_syscalls.o

```

### 3. 修改arch/riscv/include/uapi/asm/unistd.h

```
/* SPDX-License-Identifier: GPL-2.0 WITH Linux-syscall-note */
/*
 * Copyright (C) 2019 David Abdurachmanov <david.abdurachmanov@gmail.com>
 *
 * This program is free software; you can redistribute it and/or modify
 * it under the terms of the GNU General Public License version 2 as
 * published by the Free Software Foundation.
 *
 * This program is distributed in the hope that it will be useful,
 * but WITHOUT ANY WARRANTY; without even the implied warranty of
 * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
 * GNU General Public License for more details.
 *
 * You should have received a copy of the GNU General Public License
 * along with this program. If not, see <https://www.gnu.org/licenses/>.
 */

#ifndef __LP64__ || defined(__SYSCALL_COMPAT)
#define __ARCH_WANT_NEW_STAT
#define __ARCH_WANT_SET_GET_RLIMIT
#endif /* __LP64__ */

#define __ARCH_WANT_SYS_CLONE3
#define __ARCH_WANT_MEMFD_SECRET

#include <asm-generic/unistd.h>

/*
 * Allows the instruction cache to be flushed from userspace. Despite RISC-V
 * having a direct 'fence.i' instruction available to userspace (which we
 * can't trap), that's not actually viable when running on Linux because the
 * kernel might schedule a process on another hart. There is no way for
 * userspace to handle this without invoking the kernel (as it doesn't know the
 * thread-hart mappings), so we've defined a RISC-V specific system call to
 * flush the instruction cache.
 *
 * __NR_riscv_flush_icache is defined to flush the instruction cache over an
 * address range, with the flush applying to either all threads or just the
 * caller. We don't currently do anything with the address range, that's just
 * in there for forwards compatibility.
 */
#ifndef __NR_riscv_flush_icache
#define __NR_riscv_flush_icache (__NR_arch_specific_syscall + 15)
#endif
__SYSCALL(__NR_riscv_flush_icache, sys_riscv_flush_icache)

/* --- HW1 custom syscalls for RISC-V --- */
#define __NR_revstr 451
#define __NR_temdbuf 452

__SYSCALL(__NR_revstr, __riscv_sys_revstr)
__SYSCALL(__NR_temdbuf, __riscv_sys_temdbuf)

/* 保證 syscall table 長度涵蓋到 452 */
#undef __NR_syscalls
#define __NR_syscalls 453
```

### 4. 修改arch/riscv/kernel/syscall\_table.c

```
/* SPDX-License-Identifier: GPL-2.0-only
 *
 * Copyright (C) 2009 Arnd Bergmann <arnd@arndb.de>
 * Copyright (C) 2012 Regents of the University of California
 */

#include <linux/linkage.h>
#include <linux/syscalls.h>
#include <asm-generic/syscalls.h>
#include <asm/syscall.h>

asmlinkage long __riscv_sys_revstr(char __user *str, size_t n);
asmlinkage long __riscv_sys_temdbuf(int mode, void __user *data, size_t size);

#undef __SYSCALL
#define __SYSCALL(nr, call) [nr] = (call),

void * const sys_call_table[__NR_syscalls] = {
    [0 ... __NR_syscalls - 1] = sys_ni_syscall,
#include <asm/unistd.h>
};
```

### 5. 修改arch/riscv/kernel/compat\_syscall\_table.c

```
/* SPDX-License-Identifier: GPL-2.0-only

#define __SYSCALL_COMPAT

#include <linux/compat.h>
#include <linux/syscalls.h>
#include <asm-generic/mman-common.h>
#include <asm-generic/syscalls.h>
#include <asm/syscall.h>

asmlinkage long __riscv_sys_revstr(char __user *str, size_t n);
asmlinkage long __riscv_sys_temdbuf(int mode, void __user *data, size_t size);

#undef __SYSCALL
#define __SYSCALL(nr, call) [nr] = (call),

asmlinkage long compat_sys_rt_sigreturn(void);

void * const compat_sys_call_table[__NR_syscalls] = {
    [0 ... __NR_syscalls - 1] = sys_ni_syscall,
#include <asm/unistd.h>
};
```

## 6. 重新編譯

```
ubuntu@ecb5a3b62549:~/linux$ make -j"$(nproc)" ARCH=riscv CROSS_COMPILE=riscv64-linux-gnu- Image
CC      arch/riscv/kernel/asm-offsets.s
CALL    scripts/checksyscalls.sh
AS      arch/riscv/kernel/vdso/rt_sigreturn.o
CC      arch/riscv/kernel/vdso/vgettimeofday.o
AS      arch/riscv/kernel/vdso/getcpu.o
AS      arch/riscv/kernel/vdso/flush_icache.o
VDSO    arch/riscv/kernel/vdso/vdso.so.dbg
```

...

```
SORTTAB vmlinux
OBJCOPY arch/riscv/boot/Image
Kernel: arch/riscv/boot/Image is ready
```

## 7. 在 Docker 裡建立程式碼檔案(test\_revstr.c跟test\_tempbuf.c)

### test\_revstr.c:

```
ubuntu@ecb5a3b62549:~/hw1-tests$ cat > ~/hw1-tests/test_revstr.c << 'EOF'
#include <unistd.h>
#include <string.h>
#include <stdio.h>
#include <assert.h>

#define __NR_revstr 451 // 你的 sys_revstr 系統呼叫號碼

int main(void) {
    char str1[20] = "hello";
    printf("Ori: %s\n", str1);
    int ret1 = syscall(__NR_revstr, str1, strlen(str1));
    assert(ret1 == 0); // 確認回傳值為 0
    printf("Rev: %s\n", str1); // 這邊應該被反轉成 "olleh"

    char str2[20] = "Operating System";
    printf("Ori: %s\n", str2);
    int ret2 = syscall(__NR_revstr, str2, strlen(str2));
    assert(ret2 == 0);
    printf("Rev: %s\n", str2); // 這邊應該被反轉

    return 0;
}
EOF
```

### test\_tempbuf.c:

```
ubuntu@ecb5a3b62549:~/hw1-tests$ cat > ~/hw1-tests/test_tempbuf.c << 'EOF'
#include <unistd.h>
#include <stdio.h>
#include <string.h>
#include <assert.h>
#include <errno.h>

#define __NR_tempbuf 452

enum mode {
    PRINT,
    ADD,
    REMOVE
};

int main(void) {
    char buf[512];
    int ret;

    // 新增兩筆
    ret = syscall(__NR_tempbuf, ADD, "Hello", (size_t)strlen("Hello"));
    assert(ret == 0);

    ret = syscall(__NR_tempbuf, ADD, "Operating Systems", (size_t)strlen("Operating Systems"));
    assert(ret == 0);

    // 印出全部 (預期: Hello Operating Systems)
    memset(buf, 0, sizeof(buf));
    ret = syscall(__NR_tempbuf, PRINT, buf, sizeof(buf));
    assert(ret >= 0);
    printf("%s\n", buf);

    // 刪除不存在的字串 -> 應回傳 -1 並設 errno=ENOENT
    errno = 0;
    ret = syscall(__NR_tempbuf, REMOVE, "NotExist", (size_t)strlen("NotExist"));
    assert(ret == -1 && errno == ENOENT);

    // 刪除 "Hello"
    ret = syscall(__NR_tempbuf, REMOVE, "Hello", (size_t)strlen("Hello"));
    assert(ret == 0);

    // 再印一次 (預期: Operating Systems)
    memset(buf, 0, sizeof(buf));
    ret = syscall(__NR_tempbuf, PRINT, buf, sizeof(buf));
    assert(ret >= 0);
    printf("%s\n", buf);

    return 0;
}
EOF
```

交叉編譯:

```

ubuntu@ecb5a3b62549:~/hw1-tests$ ls
test_revstr.c  test_tempbuf.c
ubuntu@ecb5a3b62549:~/hw1-tests$ riscv64-linux-gnu-gcc -static -O2 -o test_revstr test_revstr.c
ubuntu@ecb5a3b62549:~/hw1-tests$ riscv64-linux-gnu-gcc -static -O2 -o test_tempbuf test_tempbuf.c
ubuntu@ecb5a3b62549:~/hw1-tests$ ls -lh
total 1.2M
-rwxrwxr-x 1 ubuntu ubuntu 574K Oct 17 11:13 test_revstr
-rw-rw-r-- 1 ubuntu ubuntu 642 Oct 17 11:13 test_revstr.c
-rwxrwxr-x 1 ubuntu ubuntu 574K Oct 17 11:13 test_tempbuf
-rw-rw-r-- 1 ubuntu ubuntu 1.2K Oct 17 11:13 test_tempbuf.c

```

把測試程式塞進 initramfs 並重新打包:

```

ubuntu@ecb5a3b62549:~$ mkdir -p initramfs && cd initramfs
ubuntu@ecb5a3b62549:~/initramfs$ ls
ubuntu@ecb5a3b62549:~/initramfs$ cd ...
-bash: cd: ...: No such file or directory
ubuntu@ecb5a3b62549:~/initramfs$ cd ..
ubuntu@ecb5a3b62549:~$ ls
hw1-tests  initramfs  initramfs.cpio.gz  linux
ubuntu@ecb5a3b62549:~$ cd initramfs
ubuntu@ecb5a3b62549:~/initramfs$ ls
ubuntu@ecb5a3b62549:~/initramfs$ gzip -dc ../initramfs.cpio.gz | cpio -idmv
.
bin
bin/busybox
home
sys
usr
usr/bin
usr/sbin
mnt
etc
init
dev
sbin
tmp
proc
proc/sys
proc/sys/kernel
3847 blocks
ubuntu@ecb5a3b62549:~/initramfs$ cp ~/hw1-tests/test_revstr ./usr/bin/
ubuntu@ecb5a3b62549:~/initramfs$ cp ~/hw1-tests/test_tempbuf ./usr/bin/
ubuntu@ecb5a3b62549:~/initramfs$ find . | cpio -o -H newc | gzip > ../initramfs.cpio.gz
6142 blocks

```

## 8. 啟動 QEMU、執行與截圖

啟動：

```
ubuntu@ecb5a3b62549:~$ qemu-system-riscv64 -lographic -machine virt \
-kernel ~/linux/arch/riscv/boot/Image \
-initrd ~/initramfs.cpio.gz \
-append "console=ttyS0 loglevel=3"

OpenSBI v1.3

Platform Name      : riscv-virtio,qemu
Platform Features  : medeleg
Platform HART Count : 1
Platform IPI Device : aclint-mswI
Platform Timer Device : aclint-mtimer @ 100000000Hz
Platform Console Device : uart8250
Platform HSM Device : ---
Platform PMU Device : ---
Platform Reboot Device : sifive_test
Platform Shutdown Device : sifive_test
Platform Suspend Device : ---
Platform CPPC Device : ---
Firmware Base      : 0x80000000
Firmware Size       : 322 KB
Firmware RW Offset : 0x40000
Firmware RW Size    : 66 KB
Firmware Heap Offset : 0x48000
Firmware Heap Size  : 34 KB (total), 2 KB (reserved), 9 KB (used), 22 KB (free)
Firmware Scratch Size : 4096 B (total), 760 B (used), 3336 B (free)
Runtime SBI Version : 1.0

Domain0 Name       : root
Domain0 Boot HART  : 0
Domain0 HARTs      : 0*
Domain0 Region00   : 0x0000000002000000-0x000000000200ffff M: (I,R,W) S/U: ( )
Domain0 Region01   : 0x00000000080040000-0x0000000008005ffff M: (R,W) S/U: ( )
Domain0 Region02   : 0x00000000080000000-0x0000000008003ffff M: (R,X) S/U: ( )
Domain0 Region03   : 0x00000000000000000-0xffffffffff M: (R,W,X) S/U: (R,W,X)
Domain0 Next Address : 0x00000000080200000
Domain0 Next Arg1   : 0x00000000087e00000
Domain0 Next Mode    : S-mode
Domain0 SysReset     : yes
Domain0 SysSuspend   : yes

Boot HART ID       : 0
Boot HART Domain    : root
Boot HART Priv Version : v1.12
Boot HART Base ISA   : rv64imafdc
Boot HART ISA Extensions : time,sstc
Boot HART PMP Count  : 16
Boot HART PMP Granularity : 4
Boot HART PMP Address Bits: 54
Boot HART MHPM Count : 16
Boot HART MIDELEG    : 0x00000000000001666
Boot HART MEDELEG    : 0x0000000000f0b509
~ #
```

在 QEMU 裡執行測試：

test\_revstr:

```
~ # /usr/bin/test_revstr
Ori: hello
Rev: olleh
Ori: Operating System
Rev: metsyS gnitarep0
```

```

~ # dmesg | tail -n 40
[ 0.288837] printk: console [ttyS0] enabled
[ 0.301825] loop: module loaded
[ 0.304733] e1000e: Intel(R) PRO/1000 Network Driver
[ 0.304748] e1000e: Copyright(c) 1999 - 2015 Intel Corporation.
[ 0.306077] usbcore: registered new interface driver uas
[ 0.306202] usbcore: registered new interface driver usb-storage
[ 0.306765] mousedev: PS/2 mouse device common for all mice
[ 0.309323] goldfish_rtc 101000.rtc: registered as rtc0
[ 0.309873] goldfish_rtc 101000.rtc: setting system clock to 2025-10-17T11:23:48 UTC (1760700228)
[ 0.311848] syscon-poweroff poweroff: pm_power_off already claimed for sbi_srst_power_off
[ 0.312248] syscon-poweroff: probe of poweroff failed with error -16
[ 0.313221] sdhci: Secure Digital Host Controller Interface driver
[ 0.313234] sdhci: Copyright(c) Pierre Ossman
[ 0.313379] sdhci-pltfm: SDHCI platform and OF driver helper
[ 0.313810] usbcore: registered new interface driver usbhid
[ 0.313828] usbhid: USB HID core driver
[ 0.314036] riscv-pmu-sbi: SBI PMU extension is available
[ 0.314400] riscv-pmu-sbi: 16 firmware and 18 hardware counters
[ 0.314423] riscv-pmu-sbi: Perf sampling/filtering is not supported as sscof extension is not available
[ 0.316532] NET: Registered PF_INET6 protocol family
[ 0.321799] Segment Routing with IPv6
[ 0.321939] In-situ OAM (IOAM) with IPv6
[ 0.322242] sit: IPv6, IPv4 and MPLS over IPv4 tunneling driver
[ 0.324470] NET: Registered PF_PACKET protocol family
[ 0.325362] 9pnet: Installing 9P2000 support
[ 0.325567] Key type dns_resolver registered
[ 0.325625] start plist test
[ 0.329247] end plist test
[ 0.330749] debug_vm_pgtable: [debug_vm_pgtable      ]: Validating architecture page table helpers
[ 0.369769] Freeing unused kernel image (initmem) memory: 2176K
[ 0.377921] Run /init as init process
[ 0.377947]   with arguments:
[ 0.377962]     /init
[ 0.377980]   with environment:
[ 0.377994]     HOME=/
[ 0.378007]     TERM=linux
[ 57.934606] The origin string: hello
[ 57.934673] The reversed string: olleh
[ 57.935103] The origin string: Operating System
[ 57.935114] The reversed string: metsyS gnitarepO
~ #

```

test\_tempbuf:

```

~ # /usr/bin/test_tempbuf
Hello Operating Systems
Operating Systems

```

```

# dmesg | tail -n 80
[ 0.136525] TCP: Hash tables configured (established 1024 bind 1024)
[ 0.137564] UDP hash table entries: 256 (order: 2, 24576 bytes, linear)
[ 0.137808] UDP-Lite hash table entries: 256 (order: 2, 24576 bytes, linear)
[ 0.138819] NET: Registered PF_UNIX/PF_LOCAL protocol family
[ 0.141444] RPC: Registered named UNIX socket transport module.
[ 0.141486] RPC: Registered udp transport module.
[ 0.141495] RPC: Registered tcp transport module.
[ 0.141502] RPC: Registered tcp NFSv4.1 backchannel transport module.
[ 0.141631] PCI: CLS 0 bytes, default 64
[ 0.147704] Unpacking initramfs...
[ 0.150813] workingset: timestamp_bits=46 max_order=15 bucket_order=0
[ 0.170066] NFS: Registering the id_resolver key type
[ 0.170889] Key type id_resolver registered
[ 0.170920] Key type id_legacy registered
[ 0.171231] nfs4filelayout_init: NFSv4 File Layout Driver Registering...
[ 0.171312] nfs4flexfilelayout_init: NFSv4 Flexfile Layout Driver Registering...
[ 0.171746] 9p: Installing v9fs 9p2000 file system support
[ 0.173341] NET: Registered PF_ALG protocol family
[ 0.173589] Block layer SCSI generic (bsg) driver version 0.4 loaded (major 250)
[ 0.173711] io scheduler mq-deadline registered
[ 0.173791] io scheduler kyber registered
[ 0.181274] pci-host-generic 30000000.pci: host bridge /soc/pci@30000000 ranges:
[ 0.181978] pci-host-generic 30000000.pci: IO 0x0003000000..0x000300ffff -> 0x0000000000
[ 0.182381] pci-host-generic 30000000.pci: MEM 0x0040000000..0x007fffffff -> 0x0040000000
[ 0.182437] pci-host-generic 30000000.pci: MEM 0x0400000000..0x07fffffff -> 0x0400000000
[ 0.182892] pci-host-generic 30000000.pci: Memory resource size exceeds max for 32 bits
[ 0.184229] pci-host-generic 30000000.pci: ECAM at [mem 0x30000000-0x3fffffff] for [bus 00-ff]
[ 0.185315] pci-host-generic 30000000.pci: PCI host bridge to bus 0000:00
[ 0.185546] pci_bus 0000:00: root bus resource [bus 00-ff]
[ 0.185625] pci_bus 0000:00: root bus resource [io 0x0000-0xffff]
[ 0.185667] pci_bus 0000:00: root bus resource [mem 0x40000000-0x7fffffff]
[ 0.185676] pci_bus 0000:00: root bus resource [mem 0x400000000-0x7fffffff]
[ 0.186808] pci 0000:00:00.0: [1b36:0008] type 00 class 0x060000
[ 0.265346] Freeing initrd memory: 1800K
[ 0.278286] Serial: 8250/16550 driver, 4 ports, IRQ sharing disabled
[ 0.285189] printk: console [ttyS0] disabled
[ 0.286836] 10000000.serial: ttyS0 at MMIO 0x10000000 (irq = 1, base_baud = 230400) is a 16550A
[ 0.288837] printk: console [ttyS0] enabled
[ 0.301825] loop: module loaded
[ 0.304733] e1000e: Intel(R) PRO/1000 Network Driver
[ 0.304748] e1000e: Copyright(c) 1999 - 2015 Intel Corporation.
[ 0.306077] usbcore: registered new interface driver uas
[ 0.306202] usbcore: registered new interface driver usb-storage
[ 0.306765] mousedev: PS/2 mouse device common for all mice
[ 0.309323] goldfish_rtc 101000.rtc: registered as rtc0
[ 0.309873] goldfish_rtc 101000.rtc: setting system clock to 2025-10-17T11:23:48 UTC (1760700228)
[ 0.311848] syscon-poweroff poweroff: pm_power_off already claimed for sbi_srst_power_off
[ 0.312248] syscon-poweroff: probe of poweroff failed with error -16
[ 0.313221] sdhci: Secure Digital Host Controller Interface driver
[ 0.313234] sdhci: Copyright(c) Pierre Ossman
[ 0.313379] sdhci-pltfm: SDHCI platform and OF driver helper
[ 0.313810] usbcore: registered new interface driver usbhid
[ 0.313828] usbhid: USB HID core driver
[ 0.314036] riscv-pmu-sbi: SBI PMU extension is available
[ 0.314400] riscv-pmu-sbi: 16 firmware and 18 hardware counters
[ 0.314423] riscv-pmu-sbi: Perf sampling/filtering is not supported as sscof extension is not available
[ 0.316532] NET: Registered PF_INET6 protocol family
[ 0.321709] Segment Routing with IPv6
[ 0.321939] In-situ OAM (IOAM) with IPv6
[ 0.322242] slt: IPv6, IPv4 and MPLS over IPv4 tunneling driver
[ 0.324470] NET: Registered PF_PACKET protocol family
[ 0.325362] 9pnet: Installing 9P2000 support
[ 0.325567] Key type dns_resolver registered
[ 0.325625] start plist test
[ 0.329247] end plist test
[ 0.330749] debug_vm_pgtable: [debug_vm_pgtable]: Validating architecture page table helpers
[ 0.369769] Freeing unused kernel image (initmem) memory: 2176K
[ 0.377921] Run /init as init process
[ 0.377947] with arguments:
[ 0.377962] /init
[ 0.377980] with environment:
[ 0.377994] HOME=/
[ 0.378007] TERM=linux
[ 57.934606] The origin string: hello
[ 57.934673] The reversed string: olleh
[ 57.935103] The origin string: Operating System
[ 57.935114] The reversed string: metsys gnitarep0
[ 192.697350] [tempbuf] Added: Hello
[ 192.697423] [tempbuf] Added: Operating Systems
[ 192.698905] [tempbuf] Removed: Hello
#

```

- **Answer the Q&As**

1. **What does the `include/linux/syscalls.h` do?**

它宣告(或由巨集生成)各個系統呼叫的「內核端原型」, 並提供 `SYSCALL_DEFINE*` 等巨集, 讓不同架構能用一致介面連到各自的 syscall table。簡單說: 這是 kernel 對「系統呼叫介面」的宣告集中地, 協助把 C 實作跟架構入口(號碼→函式)接起來。

2. **Explain the difference between a system call and a glibc library call, then give one example that maps a glibc function to the specific system call it ultimately invokes.**

- 系統呼叫(system call)是使用者空間進入核心空間的正式入口點(透過 `ecall/syscall` 指令), 由核心直接實作。
- glibc 函式是使用者態函式庫 API, 很多只是包裝(wrapper), 最終會觸發對應的系統呼叫。
- 例: `write()` 是 glibc 的函式, 會對應到內核的 `SYS_write` (`x86_64` 是號碼 1; RISC-V 也有自己的號碼表)。呼叫流程是: `write()` → 進入內核 → `sys_write` (或架構前綴名) → 返回使用者態。

3. **Explain the difference between static linking and dynamic linking**

- 靜態連結: 把需要的函式庫物件直接打包到執行檔內; 執行時不需動態載入器與共享庫檔案, 檔案較大, 但部署簡單。
- 動態連結: 執行時由動態載入器(如 `ld-linux-riscv64-lp64d.so.1`)把 `.so` 載入, 執行檔較小, 升級共享庫容易, 但執行環境必須有對應的 loader 與 `.so`。

4. **In this assignment environment, why do we have to compile the test programs with the `-static` flag? What would happen if we omitted it?**

- 用 `-static` 可以把必要函式庫全打包進二進位檔, 確保在極簡根檔系統中也能執行。
- 若省略, 執行時通常會出現 `No such file or directory` (其實是在找不到動態載入器 `.so`), 或載入失敗的錯誤, 導致程式沒辦法跑。

## 附件

### kernel/os\_hw1\_syscalls.c

```
// kernel/os_hw1_syscalls.c
#include <linux/kernel.h>
#include <linux/syscalls.h>
#include <linux/slab.h>
#include <linux/uaccess.h>
#include <linux/list.h>
#include <linux/mutex.h>
#include <linux/string.h>
#include <linux/errno.h>

/*
 * sys_revstr (nr = 451)
 * 參數: (char __user *str, size_t n)
 * 動作: 印出原字串 -> 反轉 -> 印出反轉字串 -> 寫回 user 空間 -> 回傳 0
 * 題目說可不特別處理無效引數, 但 copy_{from,to}_user 失敗時回 -EFAULT
 */
asmlinkage long __riscv_sys_revstr(char __user *str, size_t n)
{
    char *kbuf;
    size_t i = 0, j;

    if (!str || n == 0)
        return 0;

    kbuf = kmalloc(n + 1, GFP_KERNEL);
    if (!kbuf)
        return -ENOMEM;

    if (copy_from_user(kbuf, str, n)) {
        kfree(kbuf);
        return -EFAULT;
    }
    kbuf[n] = '\0';

    pr_info("The origin string: %s\n", kbuf);

    j = n - 1;
    while (i < j) {
        char t = kbuf[i];
        kbuf[i] = kbuf[j];
        kbuf[j] = t;
        i++;
        j--;
    }
}
```

```

pr_info("The reversed string: %s\n", kbuf);

if (copy_to_user(str, kbuf, n)) {
    kfree(kbuf);
    return -EFAULT;
}

kfree(kbuf);
return 0;
}

/*
 * sys_tempbuf (nr = 452)
 * 參數: (int mode, void __user *data, size_t size)
 * 模式:
 *   ADD   : 加到 list 末端, 印 [tempbuf] Added: <data>, 成功回 0
 *   REMOVE: 移除第一個匹配, 印 [tempbuf] Removed: <data>; 找不到回 -ENOENT
 *   PRINT  : 以空白串接所有字串, 複製回 data (最多 size bytes), 回傳實際複製字元數 (不含 '\0')
 * 失敗:
 *   -EFAULT: user 指標無效 (data==NULL / size==0 / copy_{from,to}_user 失敗)
 *
 * 註: 題目保證 PRINT 產出長度不會超過 512 bytes.
 */
enum mode { PRINT = 0, ADD = 1, REMOVE = 2 };

struct tb_node {
    struct list_head node;
    char *s;
    size_t len;
};

static LIST_HEAD(tb_head);
static DEFINE_MUTEX(tb_lock);

asmlinkage long __riscv_sys_tempbuf(int mode, void __user *data, size_t size)
{
    if (mode != ADD && mode != REMOVE && mode != PRINT)
        return -EINVAL;

    /* ADD / REMOVE: 先把 user 的字串複製進 kernel */
    if (mode == ADD || mode == REMOVE) {
        char *kstr;

        if (!data || size == 0)
            return -EFAULT;

        kstr = kmalloc(size + 1, GFP_KERNEL);

```

```

if (!kstr)
    return -ENOMEM;

if (copy_from_user(kstr, data, size)) {
    kfree(kstr);
    return -EFAULT;
}
kstr[size] = '\0';

if (mode == ADD) {
    struct tb_node *n = kmalloc(sizeof(*n), GFP_KERNEL);
    if (!n) {
        kfree(kstr);
        return -ENOMEM;
    }
    n->s = kstr;
    n->len = size;

    mutex_lock(&tb_lock);
    list_add_tail(&n->node, &tb_head);
    mutex_unlock(&tb_lock);

    pr_info("[tempbuf] Added: %s\n", kstr);
    return 0;
} else { /* REMOVE */
    struct tb_node *pos, *tmp;
    int found = 0;

    mutex_lock(&tb_lock);
    list_for_each_entry_safe(pos, tmp, &tb_head, node) {
        if (pos->len == size && !memcmp(pos->s, kstr, size)) {
            list_del(&pos->node);
            pr_info("[tempbuf] Removed: %s\n", pos->s);
            kfree(pos->s);
            kfree(pos);
            found = 1;
            break;
        }
    }
    mutex_unlock(&tb_lock);

    kfree(kstr);
    return found ? 0 : -ENOENT;
}
}

/* PRINT: 串接 list 內容, 以單一空白分隔, 拷回 user, 回傳拷回的實際字元數(不含 '\0') */
if (mode == PRINT) {

```

```

size_t sum = 0, cnt = 0, need, cap, written = 0;
struct tb_node *pos;
char *out;

if (!data || size == 0)
    return -EFAULT;

/* 第一次遍歷計算總長度(字串總長 + 空白數) */
mutex_lock(&tb_lock);
list_for_each_entry(pos, &tb_head, node) {
    sum += pos->len;
    cnt++;
}
mutex_unlock(&tb_lock);

if (cnt == 0) {
    /* 空清單: 回傳空字串 */
    char zero = '\0';
    if (copy_to_user(data, &zero, 1))
        return -EFAULT;
    return 0;
}

need = sum + (cnt - 1);          /* 理論需要長度(不含 '\0') */
cap = (need > 512) ? 512 : need; /* 題目保證不超過 512, 這裡仍防呆 */

out = kmalloc(cap + 1, GFP_KERNEL);
if (!out)
    return -ENOMEM;

/* 第二次遍歷實際拷貝, 注意容量 cap */
{
    size_t idx = 0;
    size_t i = 0;

    mutex_lock(&tb_lock);
    list_for_each_entry(pos, &tb_head, node) {
        if (i++ > 0) {
            if (idx < cap)
                out[idx++] = ' ';
        }
        if (idx < cap) {
            size_t can = pos->len;
            if (can > cap - idx)
                can = cap - idx;
            if (can) {
                memcpy(out + idx, pos->s, can);
                idx += can;
            }
        }
    }
}

```

```

    }
}
}
mutex_unlock(&tb_lock);

out[idx] = '\0';
written = idx; /* 不含 '\0' */
}

/* 連同結尾 '\0' 一起拷回 (但回傳值不含 '\0') */
if (copy_to_user(data, out, (written + 1 <= size) ? (written + 1) : size)) {
    kfree(out);
    return -EFAULT;
}

kfree(out);
return (int)written;
}

return 0;
}

```

## arch/riscv/include/uapi/asm/unistd.h

```

/* SPDX-License-Identifier: GPL-2.0 WITH Linux-syscall-note */
/*
 * Copyright (C) 2018 David Abdurachmanov <david.abdurachmanov@gmail.com>
 *
 * This program is free software; you can redistribute it and/or modify
 * it under the terms of the GNU General Public License version 2 as
 * published by the Free Software Foundation.
 *
 * This program is distributed in the hope that it will be useful,
 * but WITHOUT ANY WARRANTY; without even the implied warranty of
 * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
 * GNU General Public License for more details.
 *
 * You should have received a copy of the GNU General Public License
 * along with this program. If not, see <https://www.gnu.org/licenses/>.
 */

#if defined(__LP64__) && !defined(__SYSCALL_COMPAT)
#define __ARCH_WANT_NEW_STAT
#define __ARCH_WANT_SET_GET_RLIMIT
#endif /* __LP64__ */

#define __ARCH_WANT_SYS_CLONE3
#define __ARCH_WANT_MEMFD_SECRET

```

```

#include <asm-generic/unistd.h>

/*
 * Allows the instruction cache to be flushed from userspace. Despite RISC-V
 * having a direct 'fence.i' instruction available to userspace (which we
 * can't trap!), that's not actually viable when running on Linux because the
 * kernel might schedule a process on another hart. There is no way for
 * userspace to handle this without invoking the kernel (as it doesn't know the
 * thread->hart mappings), so we've defined a RISC-V specific system call to
 * flush the instruction cache.
 *
 * __NR_riscv_flush_icache is defined to flush the instruction cache over an
 * address range, with the flush applying to either all threads or just the
 * caller. We don't currently do anything with the address range, that's just
 * in there for forwards compatibility.
 */
#ifdef __NR_riscv_flush_icache
#define __NR_riscv_flush_icache (__NR_arch_specific_syscall + 15)
#endif
__SYSCALL(__NR_riscv_flush_icache, sys_riscv_flush_icache)

/* --- HW1 custom syscalls for RISC-V --- */
#define __NR_revstr 451
#define __NR_tempbuf 452

__SYSCALL(__NR_revstr, __riscv_sys_revstr)
__SYSCALL(__NR_tempbuf, __riscv_sys_tempbuf)

/* 保證 syscall table 長度涵蓋到 452 */
#undef __NR_syscalls
#define __NR_syscalls 453

```

## arch/riscv/kernel/syscall\_table.c

```

// SPDX-License-Identifier: GPL-2.0-only
/*
 * Copyright (C) 2009 Arnd Bergmann <arnd@arndb.de>
 * Copyright (C) 2012 Regents of the University of California
 */

#include <linux/linkage.h>
#include <linux/syscalls.h>
#include <asm-generic/syscalls.h>
#include <asm/syscall.h>

asmlinkage long __riscv_sys_revstr(char __user *str, size_t n);
asmlinkage long __riscv_sys_tempbuf(int mode, void __user *data, size_t size);

```

```

#undef __SYSCALL
#define __SYSCALL(nr, call)    [nr] = (call),

void * const sys_call_table[__NR_syscalls] = {
    [0 ... __NR_syscalls - 1] = sys_ni_syscall,
#include <asm/unistd.h>
};

```

## arch/riscv/kernel/compat\_syscall\_table.c

```

// SPDX-License-Identifier: GPL-2.0-only

#define __SYSCALL_COMPAT

#include <linux/compat.h>
#include <linux/syscalls.h>
#include <asm-generic/mman-common.h>
#include <asm-generic/syscalls.h>
#include <asm/syscall.h>

asmlinkage long __riscv_sys_revstr(char __user *str, size_t n);
asmlinkage long __riscv_sys_tempbuf(int mode, void __user *data, size_t size);

#undef __SYSCALL
#define __SYSCALL(nr, call)    [nr] = (call),

asmlinkage long compat_sys_rt_sigreturn(void);

void * const compat_sys_call_table[__NR_syscalls] = {
    [0 ... __NR_syscalls - 1] = sys_ni_syscall,
#include <asm/unistd.h>
};

```